

Container Overhead or Stale Dependencies? Decomposing the Latency and Energy Cost of Containerised Inference on an Edge Node

Lin Ding Shan

Abstract—Comparisons of containerised and native execution on edge devices usually change two things at once: the execution mode, and the software stack that the container image happens to carry. We separate the two on a Raspberry Pi 5 running a duty-cycled convolutional classifier with ONNX Runtime. Three conditions were measured: native execution; a container with the same interpreter and C library versions and byte-identical compiled numerical modules; and a container built from an older base image. Nine three-hour runs in a randomised complete block design recorded per-inference latency, processor load, temperature, fan speed and node input power through both phases of each duty cycle. With the stack held constant, containerisation added 1.30 ms (6.7%) to mean inference latency, 1.9% to processor time per inference and at most a few percent to energy per inference. With the execution mode held constant, the older stack added 41.5% to mean latency yet drew 6.7% less active power and used 16.3% less energy per inference above the idle floor. The idle floor accounted for 67–71% of the energy per inference over the duty cycle. Most of the apparent container cost belongs to the dependency versions inside the image.

Index Terms—Containers, edge computing, energy efficiency, performance evaluation, deep learning inference, software dependencies, measurement methodology

1 INTRODUCTION

CONTAINER runtimes have become a default way of packaging software for single-board computers at the network edge [1]–[3]. A container image fixes an application together with its interpreter, libraries and base distribution, which makes deployment repeatable [4]. It also freezes that stack at the moment the image was built. On a device that runs a neural network continuously, the frozen stack includes the inference runtime that executes the model [5].

This creates a measurement problem that the usual comparison does not address. When a containerised deployment is compared with the same application run natively, the two conditions typically differ in execution mode and in software version at once: the host has been updated and the image has not, or the image was built from whatever base tag was current. Any latency, power or thermal difference is then a joint effect, and attributing it to “container overhead” is an assumption rather than a finding. Container-versus-native studies on ARM and IoT hardware have generally reported small overheads [6]–[9]. Knoche and Eichelberger, packaging performance experiments in Docker on the Raspberry Pi, found slightly slower runs with higher variance and recommended images because they fix library versions [10]. The same property means that an image can preserve an outdated stack while the host moves on. We are not aware of a study that separates the two effects for neural-network inference and reports what each one costs in energy.

Our own earlier measurements illustrate the problem. In a first campaign on the node studied here (two native

and two containerised three-hour runs, released in [11]), the containerised configuration showed 45.6% higher mean inference latency, 9.24 percentage points lower processor utilisation, 0.469 W lower input power and a 3.71 °C lower cyclic peak temperature. Inspection of the image definition later showed that it fixed Python 3.9 and, because 1.19.2 is the last ONNX Runtime release with a wheel for that interpreter on 64-bit ARM, ONNX Runtime 1.19.2, while the host ran ONNX Runtime 1.29.0 on Python 3.13. That campaign could not say which of the two differences produced the effect, and it recorded no idle power, so it could not report energy per inference against a measured baseline.

This paper answers both questions with a three-condition design (Fig. 1). Condition A runs natively. Condition B runs in a container with the host’s interpreter and C library versions and byte-identical compiled extension modules for every imported numerical package, so $B - A$ isolates the execution mode. Condition C runs the older image, so $C - B$ isolates the software stack with the execution mode held constant. Nine three-hour runs in a randomised complete block design, with power sampled through both phases of a 60 s/15 s duty cycle, give interval estimates for both effects and an energy account against a measured idle floor.

The contributions are:

- 1) A measurement design that separates the execution-mode cost of containerisation from the cost of the software stack inside the image, with a fingerprint check that the matched container is matched.
- 2) Interval estimates for both effects on latency, processor load, power, energy and temperature for a duty-cycled inference workload on a Raspberry Pi 5.

• Lin Ding Shan is with the Faculty of Computer Science and Data Science, UCSI University, Cheras 56000, Kuala Lumpur, Malaysia.
E-mail: 1002475487@ucsiuniversity.edu.my

Containerisation itself costs 1.30 ms (6.7%) of mean latency and 1.9% of processor time per inference. The older stack accounts for 87% of the latency difference between the legacy container and native execution.

- 3) An energy account against a measured idle floor. It shows that the current stack, although faster, spends more energy per inference than the older one, and that the floor makes up 67–71% of the energy per inference over the duty cycle.
- 4) The telemetry from all nine runs, the run kit, image definitions, environment fingerprints and the analysis code, which regenerates every number and figure in this paper.

Section 2 reviews related work. Section 3 describes the node, the conditions, the instrumentation and the design. Section 4 reports the results, and Section 5 discusses what they mean for practice and where they stop.

2 RELATED WORK

2.1 Container overhead

On server hardware, containers have been found to add little CPU overhead relative to native execution, with costs concentrated in I/O and networking paths [12], [13]. Studies on ARM single-board computers and IoT devices report a similar picture for CPU-bound work [6], [7], [9] and have established that Docker-based fog deployments on the Raspberry Pi are feasible [8]. More recent work compares container engines [14] and contrasts containers with microVMs for edge I/O [15]. Energy has received less attention. Santos et al. measured the energy of workloads run inside and outside Docker on an x86 machine [16], and Raza et al. compared the performance and energy of container orchestrators on Raspberry Pi gateways [17]. These studies compare execution environments; none holds the software stack fixed while varying the execution mode for an inference workload. Knoche and Eichelberger used Docker on the Raspberry Pi to make performance experiments replicable, observed a small performance cost and higher variance, and noted that recreating a software environment is hard once particular library versions are no longer available [10]. The present design treats the stack inside the image as a factor in its own right.

2.2 Inference performance and energy at the edge

Characterisations of deep neural network inference on commercial edge devices report latency, throughput and power across devices and frameworks [18]–[20], and benchmark suites standardise the workloads and run rules [21], [22]. Energy measurement on embedded AI devices has its own metrological difficulties [23], and Raspberry Pi power has been modelled against utilisation [24]. Long-running inference on CPU-based edge devices interacts with thermal management [25]. The case for reporting the energy of machine learning, not only its accuracy, is made in [26]–[28], and the observation that idle power dominates lightly loaded systems goes back to the argument for energy-proportional computing [29]. Benchmarks usually report energy while the workload is running. A duty-cycled node also spends energy while it waits, and we report both.

2.3 Dependency staleness and measurement practice

Developers frequently leave library dependencies outdated [30], and container images on public registries often carry outdated packages with known vulnerabilities and bugs [31], [32]. Deep learning frameworks exhibit performance bugs [33], and framework versions can change model behaviour [34]. Staleness is therefore a property of real images, not a contrived condition. On method, small changes to the experimental setup can bias performance results [35]; statistically rigorous comparison requires repeated independent runs and interval estimates [36]–[38]; and repeated measurements within one run are not independent replicates [39]. We follow these practices: the replicate is the three-hour run, and inference is made across runs.

3 MEASUREMENT SYSTEM AND DESIGN

3.1 Node and workload

The node is a Raspberry Pi 5 (8 GB, four Cortex-A76 cores, 2.4 GHz maximum) with the official Active Cooler, a temperature-controlled fan. It runs Raspberry Pi OS based on Debian 13 (kernel 6.18.39) with the `ondemand` frequency governor. Power enters through a WaveShare UPS HAT (E), which supplies the board from a USB Type-C input and holds a four-cell lithium-ion pack. A USB camera delivers 640×480 frames.

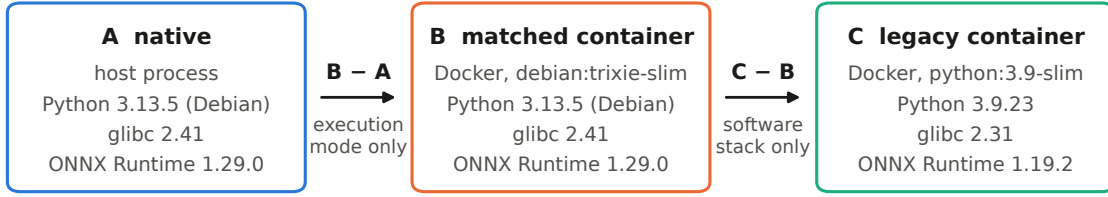
The workload is a three-class MobileNetV2 image classifier [40] built for greenhouse crop monitoring and exported to ONNX. The identical model file (SHA-256 recorded for every run) is executed by ONNX Runtime [41] on the CPU execution provider with default session options. Each frame is resized, centre-cropped to 224×224, converted to RGB and normalised; only the inference call is timed, with the wall clock. The application follows the deployed duty cycle: 60 s of continuous capture and inference, then 15 s in which the process sleeps with the camera open. The loop is camera-bound at 15 frame/s, so the number of inferences per cycle is set by the camera, not by inference speed. Each frame appends one record to a CSV log.

3.2 Conditions

Table 1 lists the three software stacks.

A (native) runs the harness directly on the host in a Python virtual environment on the distribution interpreter.

B (matched container) runs the same harness in Docker. Its image definition was generated from the host environment by a script that reads the interpreter version and the installed package versions and emits pins. It starts from `debian:trixie-slim`, installs the distribution's `python3` package, and installs the same package versions into a virtual environment. A fingerprinting script records the Python and glibc versions, the ONNX Runtime build string, and the SHA-256 of the compiled extension module of each imported numerical package (ONNX Runtime, NumPy, OpenCV, psutil). The host and B fingerprints are identical in every field. The check does not hash the interpreter binary, the C library or the shared libraries bundled with the wheels, so “matched” means the same versions of these and byte-identical extension modules, not a byte-identical file system.



A and B: same interpreter and glibc versions; compiled extension modules of onnxruntime, numpy, OpenCV and psutil byte-identical (SHA-256,

*Same board, cooler, camera, model file (SHA-256), inference code and instrumentation in all three
3 blocks × 3 conditions, order randomised within block · 3 h per run, 60 s / 15 s duty cycle*

Fig. 1. Three-condition design. $B - A$ changes only the execution mode; $C - B$ changes only the software stack inside the image. Versions are those recorded by the environment fingerprints (Table 1).

C (legacy container) runs the harness in an image built from `python:3.9-slim-bullseye`, reconstructing the first campaign’s image. Its ONNX Runtime is ten minor releases older than the host’s and is a different build; NumPy, OpenCV, psutil, Python and glibc also differ. Section 3.7 describes the reconstruction.

The containers receive the camera and the I²C bus as devices and mount the host time zone. No CPU quota was set (`cgroup cpu.max` was `max 100000` in every container run), and all four cores were available to every condition. Apart from the deviations in Section 3.7, the model, the inference code, the camera settings and the instrumentation were the same in all three.

3.3 Instrumentation

Three logs are written per run. The per-frame log records inference latency (0.1 ms resolution), system-wide CPU utilisation since the previous frame, memory utilisation, SoC temperature and the latest supply reading. A 1 Hz sampler, running in both phases of the cycle, records the UPS HAT’s Type-C bus voltage, current and power; battery voltage, current and charge; SoC temperature and CPU utilisation; the current and capped CPU frequency and the under-voltage alarm; and fan speed and PWM level. The frequency, alarm and fan values are read from sysfs in the same way in all three conditions, so that no condition carries instrumentation load that the others do not. A third log marks cycle transitions. A provenance record captures the kernel, governor, cgroup limits, CPU affinity, library versions and model hash at the start of each run.

The power figure is *node input power*: the power entering the UPS HAT’s Type-C input, which covers the board, the fan, the camera and the HAT’s own conversion losses. It is not the SoC rail. The HAT reports bus power and bus current in separate registers. Run means of bus current and of power divided by voltage agreed within 0.4% in every run; individual samples scatter more, because the registers update asynchronously. This checks the telemetry’s internal consistency, not its absolute accuracy, which the vendor does not specify. Because all conditions are measured with the same instrument and compared as differences within blocks, a constant gain or offset error largely cancels.

The charger remained connected. A run started only once the battery was floating (charge current ≤ 50 mA). Over whole runs, the mean battery current was 0.7–5.0 mA and

never left -27 to 39 mA, so charging diverted at most about 0.08 W of bus power on average.

3.4 Design and protocol

The design is a randomised complete block design with three blocks and the three conditions in each, the order within each block drawn at random with a fixed seed. A block is one session of three consecutive runs; Table 2 lists the runs in schedule order. Each run lasted three hours (144 duty cycles). The first 600 s of each run were excluded as warm-up, leaving 136 complete cycles per run for analysis. Before each run the orchestrating script waited for the SoC to cool to 50°C or below and for the battery to float, then started the run and checked its completeness and frame count afterwards.

For the whole campaign, all systemd timers were stopped so that no scheduled job could change the host stack or add load mid-run, and no interactive work was done on the node. The graphical desktop session remained running in all nine runs, at under 1% CPU. The host stack was not modified between the fingerprinting and the last run.

3.5 Energy accounting

Let $\bar{P}_{\text{act}}$ and $\bar{P}_{\text{slp}}$ be the mean node input power over the active and sleep phases, T_{act} and T_{slp} the mean phase durations (60.04 s and 15.01 s), and f the inference rate during the active phase. The energy the node spends per inference over the duty cycle is

$$E_{\text{cyc}} = \frac{\bar{P}_{\text{act}}T_{\text{act}} + \bar{P}_{\text{slp}}T_{\text{slp}}}{fT_{\text{act}}}. \quad (1)$$

The sleep phase does not reach its floor immediately (Fig. 2). The *ondemand* governor kept the maximum frequency for about 5 s after each active phase ended, and the node drew 0.46 – 0.93 W more during those seconds than later in the sleep phase. We therefore define the settled floor P_{floor} as the mean power over sleep-phase samples taken at least 6 s after the phase began, and split E_{cyc} into a baseline and a workload increment:

$$E_{\text{base}} = \frac{P_{\text{floor}}(T_{\text{act}} + T_{\text{slp}})}{fT_{\text{act}}}, \quad (2)$$

$$E_{\text{inc}} = E_{\text{cyc}} - E_{\text{base}}. \quad (3)$$

TABLE 1

Software stacks as recorded by the environment fingerprints. A and B agree in every recorded field, including the SHA-256 of each compiled extension module. The legacy ONNX Runtime build string records its compiler flags; the host build string records none.

	A native	B matched container	C legacy container
Base	host (Debian 13)	debian:trixie-slim	python:3.9-slim-bullseye (Debian 11)
Python	3.13.5 (Debian package)	3.13.5 (Debian package)	3.9.23 (image build)
glibc	2.41	2.41	2.31
ONNX Runtime	1.29.0 (build 2e2543fbe9)	1.29.0 (build 2e2543fbe9)	1.19.2 (build ffeed9d44)
NumPy / OpenCV (headless)	2.5.2 / 5.0.0.93	2.5.2 / 5.0.0.93	1.26.4 / 4.10.0.84
psutil / smbus2	7.2.2 / 0.6.1	7.2.2 / 0.6.1	6.1.0 / 0.5.0

TABLE 2

Run schedule. Order within each block was randomised. Start times are local (UTC+8). Camera: frame rate held by the camera in low light (dyn.) or requested and pinned (pinned); see Section 3.7.

Block	Run	Cond.	Start	Inferences	frame/s	Camera
1	1	C	27 Sep 03:56	130,093	15.05	dyn.
1	2	B	27 Sep 06:57	130,054	15.05	dyn.
1	3	A	27 Sep 20:19	128,922	14.91	pinned
2	1	C	27 Sep 11:15	128,865	14.91	pinned
2	2	A	27 Sep 14:16	128,923	14.91	pinned
2	3	B	27 Sep 17:17	128,895	14.91	pinned
3	1	B	27 Sep 23:22	128,788	14.90	pinned
3	2	C	28 Sep 02:23	128,573	14.88	pinned
3	3	A	28 Sep 05:24	128,577	14.88	pinned

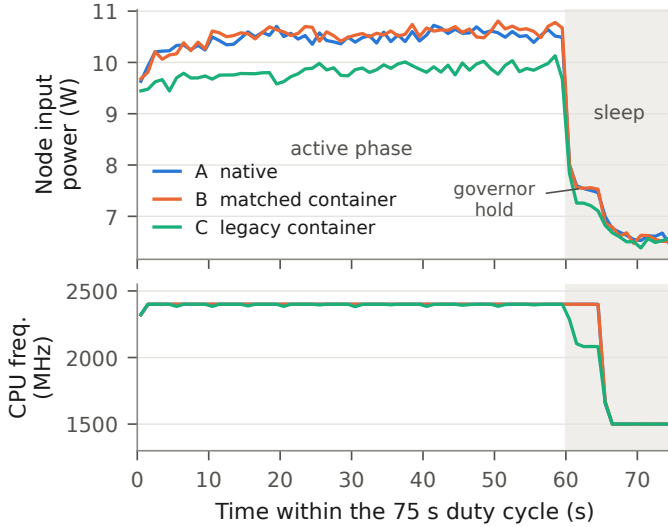


Fig. 2. Mean node input power (top) and CPU frequency (bottom) by second of the 75 s duty cycle, averaged over the analysed cycles of the three runs of each condition. Shaded: sleep phase. After each active phase the *ondemand* governor holds the maximum frequency for about 5 s before stepping down to 1500 MHz; the settled floor is the mean over sleep-phase samples from 6 s onward.

E_{base} is what the node would spend per inference if it drew only its floor for the whole cycle; E_{inc} is the energy above that floor, including the governor's hold after each active phase. The floor is not a hardware idle state. The process stays resident with the camera open and the desktop running, so P_{floor} is the node's floor under this software configuration, not the hardware's minimum.

3.6 Statistical analysis

The unit of replication is the run. For each quantity, a run-level value is computed over the post-warm-up period. An effect such as $B - A$ is estimated as the mean of the three within-block differences, with a 95% confidence interval from the t distribution with two degrees of freedom. With three blocks the intervals are wide, and we report them rather than p -values, together with the sign of the difference in each block. Percentages are relative to the reference condition of each contrast (A for $B - A$, B for $C - B$). Mean latency and energy per inference above the floor and over the cycle are the quantities of primary interest; the others are reported for completeness, and their intervals are not adjusted for multiplicity.

3.7 Deviations from protocol

Four events are reported so that readers can judge them.

Camera frame rate. The camera's default exposure mode lowers its frame rate in low light. The first two runs (*legacy_1* and *matched_1*) ran in a dark room and held 15.0 frame/s in every hour. In the next run, the original *native_1*, the operator saw the frame rate rise when the room lights came on; the run was stopped and its partial logs were not kept. From the following run (*legacy_2*) onward, dynamic frame rate was disabled on the host before every run, 15 frame/s was requested explicitly, and the camera controls were saved with each run.

Harness update and image rebuild. To request the frame rate, the harness was updated after the second run; the change also added a measured-rate line to the run summary and did not touch the preprocessing, the timed inference call or the per-frame logging (the earlier version of the harness was not retained). Both images were rebuilt to include the new harness. The harness is copied in the images' final layer, after the layers that install the software stack, so unless the build cache is bypassed a rebuild reuses the stack layers. The binary fingerprints were taken before the first run and were not repeated after the rebuild, but the per-run provenance records show the same interpreter, C library, ONNX Runtime, NumPy and OpenCV versions in every run of each condition. Within block 1, B ran with the camera unpinned at 15.05 frame/s and A with the camera pinned.

Block 1 timing. The replacement *native_1* ran in the evening of the same day (20:19–23:19 local time), after block 2, and was not contiguous with the other block 1 runs. Section 4.5 repeats the analysis without block 1.

Legacy image reconstruction. The first campaign's image no longer existed, so condition C was rebuilt from its definition. That definition did not pin package versions. The base

image fixes Python 3.9, and ONNX Runtime 1.19.2 is the only release that fits it, being the last with a wheel for that interpreter on 64-bit ARM. The other packages were pinned to the versions in Table 1; it cannot be shown that the original build resolved the same versions. One system package in the definition (`libglib2.0-0`) is no longer available from the base distribution's archive and was omitted; the headless OpenCV build does not require it. Condition C therefore reproduces the first campaign's Python minor version and runtime, but it is not a copy of that image.

Run start times, per-run tables and fingerprints are given in the supplemental material.

4 RESULTS

4.1 Run integrity

All nine runs completed, each with 128,573–130,093 inferences (1,161,690 in total). The inference rate was 14.88–14.91 frame/s in the seven runs with the camera pinned and 15.05 frame/s in the two early runs. No 1 Hz sample showed a CPU frequency cap below 2400 MHz (0 samples) or an under-voltage alarm (0 samples), and the mean active-phase frequency was at least 2393 MHz in every run. The highest SoC temperature recorded in any run was 68.8 °C. Every run used the same model file. Table 3 gives the condition means and both contrasts; Fig. 4 shows the contrasts as percentages.

4.2 Latency

Mean inference latency was 19.37, 20.68 and 29.26 ms in A, B and C. The containerisation effect $B - A$ was 1.30 ms (6.7%, 95% CI 0.52 to 2.08 ms) and positive in all three blocks. The stack effect $C - B$ was 8.58 ms (41.5%, CI 5.81 to 11.35 ms), also positive in every block. Of the total difference between the legacy container and native execution, $C - A = 9.89$ ms, the execution mode accounts for 13% and the stack for 87%.

The distributions (Fig. 3) show that the two effects differ in kind. A and B share the same fast mode: their per-run first percentiles were 14.7–15.0 and 14.7–14.8 ms, and their fastest inferences 14.6 and 14.6 ms. What differs is how often an inference falls in that mode: 40–43% of inferences took under 17.5 ms in the native runs against 19–26% in the matched-container runs. Containerisation did not slow the fastest inferences; it made fast inferences less frequent. The legacy stack shifted the whole distribution: its first percentile was 24.5–24.6 ms and its fastest inference 24.4 ms. At the tail, $B - A$ was not resolved (p95 +0.20 ms, CI –3.30 to 3.70; p99 +2.55 ms, CI –1.79 to 6.90), whereas $C - B$ was 15.37 ms at p95 (58.7%) and 12.38 ms at p99.

4.3 Processor load, power and energy

Containerisation raised processor load slightly. System-wide CPU utilisation was 1.3 percentage points higher in B than in A, positive in all three blocks but with an interval that includes zero (–0.1 to 2.6). Expressed as CPU time per inference (utilisation $\times$ four cores / inference rate), which removes the small frame-rate differences between runs, the effect was 2.8 ms (1.9%, CI 0.9 to 4.8 ms). Power did not change detectably: $B - A$ was +0.06 W in the active phase (CI –0.08 to 0.19) and –0.02 W in the sleep phase. Energy

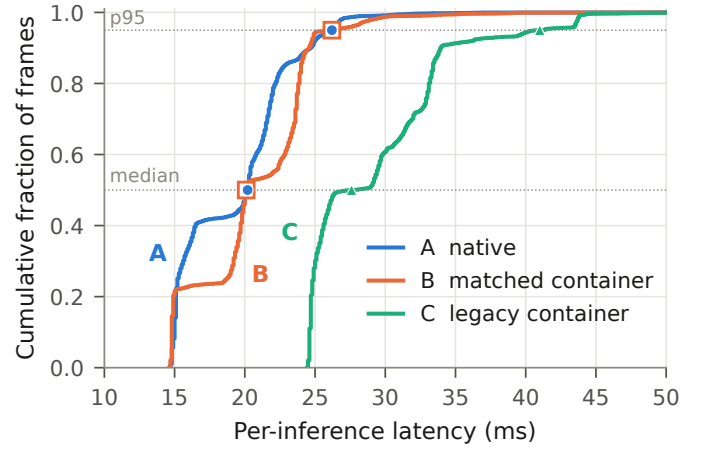


Fig. 3. Empirical distribution of per-inference latency, pooled over the three runs of each condition after warm-up. Markers show the pooled median and 95th percentile; B's markers are drawn hollow because they coincide with A's.

per inference above the floor was +4.0 mJ (+1.5%, CI –2.4% to 5.5%), and over the cycle +0.7 mJ (+0.1%, CI –2.3% to 2.4%). Effects on cycle energy of more than a few percent are therefore excluded; smaller ones are not.

The stack effect runs against the latency result. With the older stack, CPU utilisation was 8.8 percentage points lower (–15.3%, CI –10.8 to –6.8) and CPU time per inference 23.4 ms lower (–15.2%). Active-phase power was 0.70 W lower (–6.7%, CI –1.20 to –0.20 W), while the settled floor did not differ detectably (–0.06 W). Energy per inference above the floor, Eq. (3), was 43.8 mJ lower (–16.3%, CI –60.0 to –27.5 mJ, negative in every block), and energy per inference over the cycle, Eq. (1), was 48.6 mJ lower (–5.9%, CI –89.8 to –7.5 mJ). Relative to the legacy stack, then, the current stack takes 29% less time per inference call and spends 20% more energy above the floor on each inference. It also uses about 23 ms more CPU time per inference while finishing the timed call about 8.6 ms sooner, which is consistent with more cores being busy during each call. This CPU time covers the whole process and the system, not only the inference call. Fig. 5 shows the trade-off run by run: the three legacy runs form a separate cluster with longer latency, less CPU time and less energy per inference, while the native and matched runs overlap, B lying slightly above A in CPU time.

Part of the power difference is the fan. The fan draws from the same supply, and in blocks 1 and 3 the legacy run's fan was slower than the matched run's, by 1401 and 818 rpm. In block 2, however, where the legacy fan ran 80 rpm faster, active power was still 0.47 W lower, so the power difference is not explained by the fan alone.

Fig. 2 shows where in the cycle the power difference arises. The legacy stack's active-phase power sits below that of A and B for the whole 60 s, not only at the start of each phase, so the difference is a property of steady-state inference rather than of start-up. After the active phase, all three conditions settle to the same floor within a few seconds. In condition C the governor stepped down to a lower frequency sooner in part of the cycles, so its hold after the active phase cost slightly less energy; this is included in

TABLE 3

Condition means ($\pm$ SD over three runs) and within-block effects with 95% confidence intervals (t , 2 d.f.). $\dagger$: interval includes zero. Signs: direction of the difference in blocks 1, 2 and 3. CPU utilisation is system-wide, sampled per frame in the active phase. The settled floor is the sleep-phase power after the governor has stepped down; energy per inference above the floor and over the whole duty cycle are defined in Section 3.5.

Quantity	Condition mean $\pm$ SD			$B - A$ (execution mode)			$C - B$ (software stack)		
	A native	B matched	C legacy	effect	95% CI	signs	effect	95% CI	signs
Inference rate (frame/s)	14.90 $\pm$ 0.02	14.95 $\pm$ 0.09	14.94 $\pm$ 0.09	+0.05 †	[-0.14, 0.24]	+++	-0.01 †	[-0.03, 0.01]	---
Latency, mean (ms)	19.37 $\pm$ 0.14	20.68 $\pm$ 0.41	29.26 $\pm$ 0.82	+1.30	[0.52, 2.08]	+++	+8.58	[5.81, 11.35]	+++
Latency, median (ms)	20.20 $\pm$ 0.17	20.83 $\pm$ 1.44	27.10 $\pm$ 2.31	+0.63 †	[-2.76, 4.02]	+0-	+6.27 †	[-2.30, 14.84]	+++
Latency, p95 (ms)	26.00 $\pm$ 0.61	26.20 $\pm$ 1.04	41.57 $\pm$ 1.96	+0.20 †	[-3.30, 3.70]	---	+15.37	[8.10, 22.64]	+++
Latency, p99 (ms)	28.90 $\pm$ 0.80	31.45 $\pm$ 1.51	43.83 $\pm$ 0.15	+2.55 †	[-1.79, 6.90]	+++	+12.38	[8.98, 15.78]	+++
CPU utilisation, system (%)	56.1 $\pm$ 0.2	57.3 $\pm$ 0.7	48.6 $\pm$ 0.4	+1.3 †	[-0.1, 2.6]	+++	-8.8	[-10.8, -6.8]	---
CPU time per inference (ms)	150.6 $\pm$ 0.4	153.4 $\pm$ 0.9	130.0 $\pm$ 1.4	+2.8	[0.9, 4.8]	+++	-23.4	[-28.4, -18.3]	---
Power, active phase (W)	10.46 $\pm$ 0.08	10.51 $\pm$ 0.03	9.81 $\pm$ 0.19	+0.06 †	[-0.08, 0.19]	+++	-0.70	[-1.20, -0.20]	---
Power, sleep phase (W)	6.97 $\pm$ 0.05	6.95 $\pm$ 0.02	6.81 $\pm$ 0.20	-0.02 †	[-0.14, 0.11]	---	-0.14 †	[-0.67, 0.39]	+-
Power, settled floor (W)	6.61 $\pm$ 0.06	6.60 $\pm$ 0.05	6.54 $\pm$ 0.18	-0.02 †	[-0.11, 0.08]	---	-0.06 †	[-0.53, 0.41]	+-
Energy/inference above floor (mJ)	263.9 $\pm$ 1.1	267.9 $\pm$ 3.1	224.2 $\pm$ 9.5	+4.0 †	[-6.3, 14.4]	---	-43.8	[-60.0, -27.5]	---
Energy/inference, cycle (mJ)	818.8 $\pm$ 5.4	819.5 $\pm$ 2.4	770.9 $\pm$ 17.7	+0.7 †	[-18.6, 20.0]	---	-48.6	[-89.8, -7.5]	---
Cyclic peak temperature ($^\circ$ C)	64.9 $\pm$ 1.5	64.3 $\pm$ 0.3	61.6 $\pm$ 1.9	-0.6 †	[-3.6, 2.4]	---	-2.7 †	[-7.9, 2.5]	---
Fan speed, active phase (rpm)	5336 $\pm$ 192	5234 $\pm$ 69	4521 $\pm$ 712	-101 †	[-454, 252]	---	-713 †	[-2567, 1140]	+-

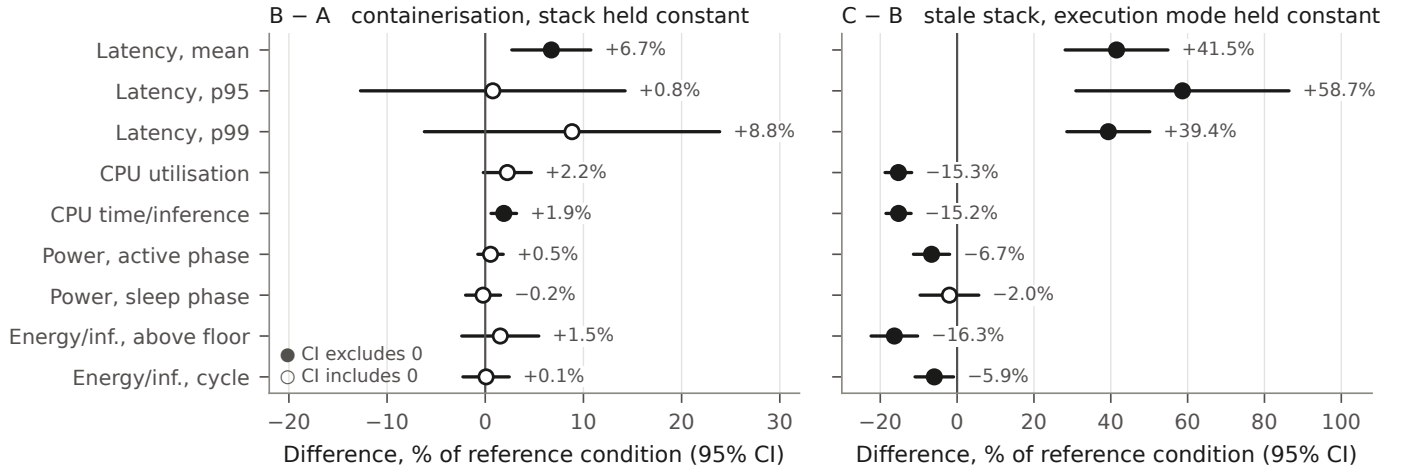


Fig. 4. The two contrasts as percentages of the reference condition, with 95% confidence intervals from three within-block differences (t , 2 d.f.). Filled markers: interval excludes zero. CPU time per inference is system-wide CPU utilisation $\times$ four cores / inference rate.

E_{inc} .

Fig. 6 splits energy per inference over the cycle into the baseline, Eq. (2), and the increment. The baseline was 554.9, 551.6 and 546.7 mJ in A, B and C, or 68, 67 and 71% of the total. The resolved part of the stack effect lies in the increment, which carries 90% of the $C - B$ difference in cycle energy.

4.4 Temperature and fan

Mean cyclic peak SoC temperature was 64.9, 64.3 and 61.6 $^\circ$ C (Fig. 7). Neither contrast was resolved. $B - A$ was -0.6 $^\circ$ C (CI -3.6 to 2.4). $C - B$ was -2.7 $^\circ$ C (CI -7.9 to 2.5); it was negative in all three blocks but ranged from -0.4 to -4.4 $^\circ$ C. Active-phase fan speed moved the same way on average ($C - B$ -713 rpm, CI -2567 to 1140). The fan controller regulates temperature, so part of any difference in heat output appears as fan speed rather than as temperature, and the two readings must be read together. In block 2 the legacy run's fan ran slightly faster than the matched run's, and its peak temperature was within 0.4 $^\circ$ C of it.

4.5 Sensitivity and relation to the first campaign

Excluding block 1, whose native run was repeated later and whose matched run had the camera unpinned, leaves two blocks. The point estimates change little: $B - A$ latency +1.14 ms and $C - B$ latency +9.15 ms (+1.30 and +8.58 ms with all blocks); $C - B$ energy per inference above the floor -40.1 mJ (-43.8). With two blocks the intervals use $t_{0.975,1} = 12.7$ and widen sharply: the $B - A$ latency interval then includes zero (-0.45 to 2.72 ms), while the $C - B$ intervals for latency (2.42 to 15.88 ms) and energy above the floor (-61.3 to -18.9 mJ) still exclude it. One estimate sharpens. In blocks 2 and 3, $B - A$ energy above the floor was +6.4 mJ (+2.4%), and its two-block interval (1.2 to 11.6 mJ) excludes zero. Block 1 is the only block with a negative $B - A$ difference in this quantity (-0.8 mJ), and it is the block in which B ran with the camera unpinned at a slightly higher frame rate. Containerisation therefore probably costs a small amount of workload energy, of the order of 2% of the energy above the floor, or +0.6% of the energy per inference over the cycle.

The first campaign [11] used an older harness with a lower frame rate, two runs per condition and no idle mea-

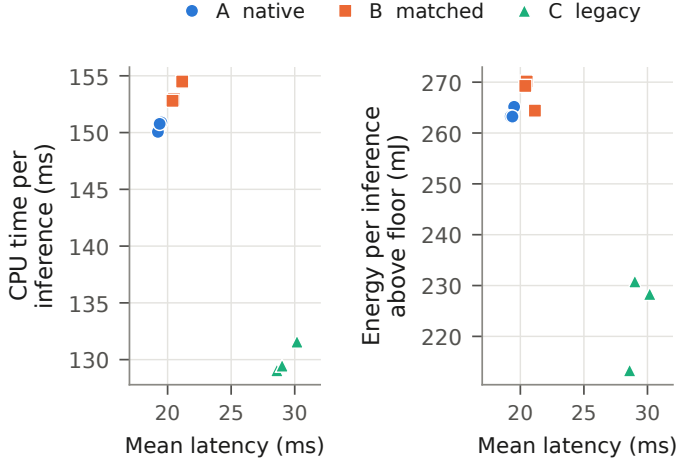


Fig. 5. Speed against cost for each of the nine runs: mean inference latency against CPU time per inference (left) and against energy per inference above the settled floor (right). The current stack (A, B) is faster but spends more CPU time and more energy on each inference than the legacy stack (C).

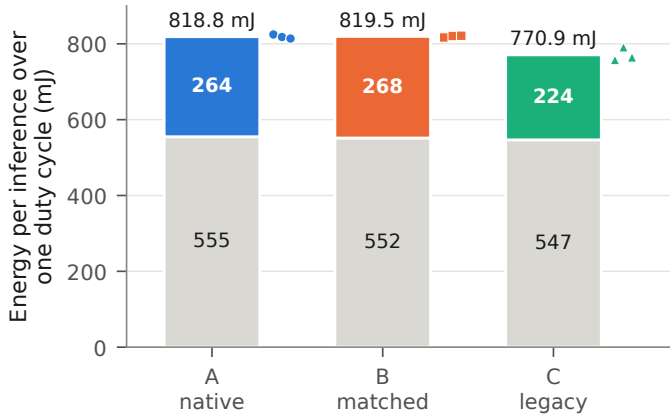


Fig. 6. Energy per inference over one 60 s/15 s duty cycle, Eq. (1). Grey: baseline at the settled sleep-phase floor, Eq. (2); colour: increment above the floor, Eq. (3); markers: individual runs. Totals are condition means.

surement, so its values are not pooled with these. Its contrast corresponds to $C - A$ here, and every sign reproduces: mean latency +51.0% (first campaign +45.6%), CPU utilisation 7.5 percentage points lower (9.24), active power 0.64 W lower (0.469 W) and cyclic peak temperature 3.3 °C lower (3.71 °C). The difference is in interpretation: the present design shows that most of that contrast comes from the stack in the image, not from running in a container.

5 DISCUSSION

5.1 What a container-versus-native figure measures

Unless the stacks are verified equal, a comparison between a containerised deployment and native execution measures two things. On this node, the execution-mode component is small: 1.30 ms of mean latency, 1.9% of CPU time per inference, no detectable change in power or temperature, and probably about 2% of the energy above the floor. The software-stack component is about 6.6 times larger in latency and determines the energy result. A study that reports

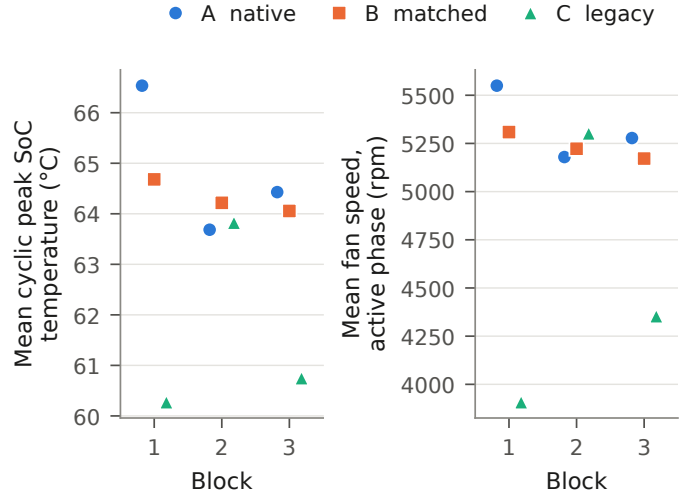


Fig. 7. Mean cyclic peak SoC temperature (left) and mean active-phase fan speed (right) for each run, by block. The Active Cooler's fan is temperature-controlled, so the two panels are coupled.

the joint figure as container overhead will overstate it; one that happened to package a newer stack than the host's could understate it or reverse its sign. The practical requirement is cheap. Record the interpreter, the C library, and the versions and hashes of the numerical binaries in each condition, and either match them or report the mismatch as a factor. The fingerprinting and Dockerfile-generation scripts released with this paper do this in seconds; hashing the interpreter, the C library and the bundled shared libraries as well would make the check stricter.

The containerisation effect that remains is informative. It did not raise the latency floor, which is inconsistent with a constant cost added to every call. Instead, it lowered the frequency of the fast mode. A change in how often the fast path is taken is more consistent with an interaction with scheduling, core placement or cache state than with added instructions on every call. The present measurements cannot identify it. Hardware performance counters and scheduler tracing, which we did not collect so as to keep the instrumentation light and identical in all conditions, are the tools for that question.

5.2 A faster stack is not a cheaper one

The current stack finished each inference call in 29% less time than the older one and spent more energy doing so. The loop is camera-bound, so finishing sooner does not produce more inferences; it only lengthens the wait before the next frame. Whether a faster inference saves energy then depends on how much power is drawn while it runs and whether the cores return to a low-power state between frames. Here the current stack drew more power during the active phase than its shorter calls could repay, consistent with its higher CPU time per inference. Updating the software stack to reduce latency can therefore increase the energy of a rate-limited edge node. Conversely, a stale image can look more energy-efficient than a current one for reasons that have nothing to do with containerisation. Energy comparisons across deployment configurations should state the runtime and library versions and, for rate-limited

loops, report energy per inference and not latency alone. The $C - B$ contrast changes the whole stack, so the present data do not say which component is responsible. Runtime session options, such as the number of intra-operator threads and whether idle worker threads spin, are candidates that a follow-up study can vary directly, alongside the library versions.

5.3 The idle floor dominates

On this node, 67–71% of the energy per inference over the duty cycle is accounted for by the settled sleep-phase floor drawn over the whole cycle, and that share hardly depended on the condition. The largest effect on energy per inference over the cycle, the stack effect, was 5.9%. For sustainability, the software stack matters, but the platform's floor matters more: the desktop session, the open camera, the fan and the conversion losses of the supply board contribute to it. Reducing that floor, for example by running headless, releasing the camera during the wait, or entering a deeper idle state, is a larger lever than any choice among the three conditions studied here, although we did not measure these alternatives. The governor's hold at maximum frequency after each active phase cost about 5 mJ per inference, small by comparison. Over a year of continuous operation on this duty cycle, the node draws about 86, 86 and 81 kWh in conditions A, B and C, of which about 58 kWh is the settled floor alone. The stack effect is worth about 5.2 kWh per node per year (CI -9.5 to -0.8 kWh), containerisation itself about 0.4 kWh (CI -0.8 to 1.5 kWh). Per node these are small amounts; across a fleet of devices kept on an outdated image they add up, but they remain small beside the roughly 58 kWh that the floor itself accounts for. This is the energy-proportionality argument [29] at the scale of a single edge node. It also means that energy figures reported without the idle phase describe only the smaller part of what a duty-cycled node consumes.

5.4 Thermal effects under active cooling

With a temperature-controlled fan, a difference in heat output shows up partly as a temperature difference and partly as a fan-speed difference. The older stack's lower power was accompanied by lower cyclic peaks and a slower fan in blocks 1 and 3, and by a slightly faster fan with almost no temperature difference in block 2. The within-block order also matters here. In blocks 1 and 3 the legacy runs fell in the pre-dawn hours, which are usually the coolest, whereas in block 2 the legacy run was at midday. Ambient temperature was not logged, so an ambient contribution to the thermal and fan differences in blocks 1 and 3 cannot be excluded. For thermal comparisons on actively cooled boards, temperature alone can hide a real difference in heat, and the fan's state and the ambient temperature should be logged alongside it.

5.5 Recommendations for measuring containerised edge inference

The results suggest five practices for comparisons of this kind. First, fingerprint the software stack in every condition, including interpreter, C library, runtime build and the

hashes of compiled modules, and include a version-matched container so that the execution mode can be separated from the stack. Second, treat the run, not the frame, as the replicate: repeat runs, block them by session so that time-of-day and thermal history cancel, and report intervals across runs. Third, log power through the idle phase of the duty cycle and report energy per inference both over the cycle and above a settled floor; energy measured only while the workload runs omits most of what a duty-cycled node consumes. Fourth, on actively cooled boards, log fan state and ambient temperature alongside SoC temperature, because a temperature-controlled fan converts heat differences into speed differences. Fifth, for rate-limited loops, report energy per inference and CPU time per inference, not latency alone, since a faster call can cost more energy. All five are cheap; the scripts released with this paper implement the first three.

5.6 Limitations

The results come from one board, one model, one pair of software stacks and one duty cycle, and they should be read as a controlled case study, not as general constants. Three blocks give wide intervals: the $B - A$ energy and thermal contrasts are consistent with zero over all blocks, and the small $B - A$ energy effect suggested by blocks 2 and 3 rests on two blocks. The $C - B$ contrast bundles the whole stack; the inference call is the timed region, which points towards the runtime, but Python, NumPy, OpenCV and glibc also changed and were not varied separately. ONNX Runtime thread counts and spinning behaviour were left at their defaults and not recorded at run time. Power is node input power sampled at 1 Hz from a module with unspecified absolute accuracy; differences between conditions are more trustworthy than absolute values, and the fan's share of the power difference was not measured separately. Ambient temperature was not logged; the blocks remove variation between sessions, not within them. The floor is not a hardware idle state, and the governor was `ondemand` throughout; a different governor or a deeper idle state would change the baseline and possibly the stack effect. Block 1 was not contiguous and ran B with the camera unpinned. The images were rebuilt once during the campaign, and the legacy image was reconstructed from its definition rather than reused.

6 CONCLUSION

We separated the cost of running an edge inference workload in a container from the cost of the software stack that the container carried. With the interpreter, C library and numerical extension modules held the same, containerisation added 1.30 ms (6.7%) to mean inference latency and 1.9% to CPU time per inference, no resolvable change in power or temperature, and probably about 2% to the energy above the idle floor. The older stack found in a real image accounted for 87% of the latency difference, and it was the less energy-hungry configuration: 16.3% less energy per inference above the idle floor and 5.9% less over the duty cycle, with lower active power in every block. The idle floor made up 67–71% of the energy per inference. Container-versus-native comparisons at the edge should therefore verify and report

the software stack in each condition, and energy studies of duty-cycled nodes should measure the idle phase. Future work will use hardware performance counters and scheduler tracing to explain the change in the fast-mode frequency under containerisation, and will vary runtime threading options and library versions separately to locate the energy difference between the stacks.

DATA AND CODE AVAILABILITY

The per-frame, 1 Hz and cycle-event logs of all nine runs, with their provenance records, camera settings, environment fingerprints and orchestration log, are deposited at Zenodo [42]; the first campaign's telemetry is in version 1.0.0 of the same record [11]. The run kit (harness, orchestrator, image definitions and fingerprinting script) and the analysis and figure scripts, which regenerate every number, table and figure in this paper from the raw logs, are at <https://github.com/xiaolin200206/edge-container-deployment-measurement> (release v2.0-tsusc).

ACKNOWLEDGMENT

The author built the edge node, carried out all measurements, and is responsible for the analysis, its interpretation and the conclusions. In accordance with IEEE policy, the author discloses that a generative AI tool, Claude (Anthropic), was used as an assistant in refining the experimental protocol, in writing and debugging parts of the measurement, orchestration and analysis scripts, in drafting and language editing of the manuscript text, and in checking reference metadata. All AI-assisted material was reviewed, tested against the raw data and revised by the author.

REFERENCES

- [1] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, 2014.
- [2] C. Pahl, A. Brogi, J. Soldani, and P. Jamshidi, "Cloud container technologies: A state-of-the-art review," *IEEE Transactions on Cloud Computing*, vol. 7, no. 3, pp. 677–692, 2019.
- [3] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [4] C. Boettiger, "An introduction to Docker for reproducible research," *ACM SIGOPS Operating Systems Review*, vol. 49, no. 1, pp. 71–79, 2015.
- [5] J. Chen and X. Ran, "Deep learning with edge computing: A review," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1655–1674, 2019.
- [6] R. Morabito, "A performance evaluation of container technologies on Internet of Things devices," in *Proc. IEEE Conf. Computer Communications Workshops (INFOCOM WKSHPS)*, 2016, pp. 999–1000.
- [7] —, "Virtualization on Internet of Things edge devices with container technologies: A performance evaluation," *IEEE Access*, vol. 5, pp. 8835–8850, 2017.
- [8] P. Bellavista and A. Zanni, "Feasibility of fog computing deployment based on Docker containerization over RaspberryPi," in *Proc. 18th Int. Conf. Distributed Computing and Networking (ICDCN)*, ACM, 2017.
- [9] F. Ramalho and A. Neto, "Virtualization at the network edge: A performance comparison," in *Proc. IEEE 17th Int. Symp. A World of Wireless, Mobile and Multimedia Networks (WoWMoM)*, 2016, pp. 1–6.
- [10] H. Knoche and H. Eichelberger, "Using the Raspberry Pi and Docker for replicable performance experiments: Experience paper," in *Proc. ACM/SPEC Int. Conf. Performance Engineering (ICPE)*, 2018, pp. 305–316.
- [11] D. S. Lin, "Greenhouse imagery and deployment telemetry for a containerised edge vision node on a Raspberry Pi 5 (version 1.0.0)," Zenodo dataset, 2026. [Online]. Available: <https://doi.org/10.5281/zenodo.22857312>
- [12] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, "An updated performance comparison of virtual machines and Linux containers," in *Proc. IEEE Int. Symp. Performance Analysis of Systems and Software (ISPASS)*, 2015, pp. 171–172.
- [13] R. Morabito, J. Kjällman, and M. Komu, "Hypervisors vs. lightweight virtualization: A performance comparison," in *Proc. IEEE Int. Conf. Cloud Engineering (IC2E)*, 2015, pp. 386–393.
- [14] L. Baresi, G. Quattrocchi, and N. Rasi, "A qualitative and quantitative analysis of container engines," *Journal of Systems and Software*, vol. 210, p. 111965, 2024.
- [15] K. Lee, Y. Choi, and B. Tak, "Performance analysis of microVMs and containers for edge computing: A focus on file and network I/O," *Future Generation Computer Systems*, vol. 176, p. 108176, 2026.
- [16] E. A. Santos, C. McLean, C. Solinas, and A. Hindle, "How does Docker affect energy consumption? Evaluating workloads in and out of Docker containers," *Journal of Systems and Software*, vol. 146, pp. 14–25, 2018.
- [17] S. M. Raza, J. Jeong, M. Kim, B. Kang, and H. Choo, "Empirical performance and energy consumption evaluation of container solutions on resource constrained IoT gateways," *Sensors*, vol. 21, no. 4, p. 1378, 2021.
- [18] R. Hadidi, J. Cao, Y. Xie, B. Asgari, T. Krishna, and H. Kim, "Characterizing the deployment of deep neural networks on commercial edge devices," in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, 2019, pp. 35–48.
- [19] Q. Liang, P. Shenoy, and D. Irwin, "AI on the edge: Characterizing AI-based IoT applications using specialized edge architectures," in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, 2020, pp. 145–156.
- [20] S. P. Baller, A. Jindal, M. Chadha, and M. Gerndt, "DeepEdgeBench: Benchmarking deep neural networks on edge devices," in *Proc. IEEE Int. Conf. Cloud Engineering (IC2E)*, 2021, pp. 20–30.
- [21] V. J. Reddi et al., "MLPerf inference benchmark," in *Proc. ACM/IEEE 47th Annu. Int. Symp. Computer Architecture (ISCA)*, 2020, pp. 446–459.
- [22] C. Banbury et al., "MLPerf tiny benchmark," in *Proc. Neural Information Processing Systems Track on Datasets and Benchmarks*, vol. 1, 2021. [Online]. Available: <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/da4fb5c6e93e74d3df8527599fa62642-Abstract-round1.html>
- [23] A. Apicella et al., "Energy consumption assessment in embedded AI: Metrological improvements of benchmarks for edge devices," *Computer Standards & Interfaces*, vol. 97, p. 104120, 2026.
- [24] F. Kaup, P. Gottschling, and D. Hausheer, "PowerPi: Measuring and modeling the power consumption of the Raspberry Pi," in *Proc. 39th Annu. IEEE Conf. Local Computer Networks (LCN)*, 2014, pp. 236–243.
- [25] T. Benoit-Cattin, D. Velasco-Montero, and J. Fernández-Berni, "Impact of thermal throttling on long-term visual inference in a CPU-based edge device," *Electronics*, vol. 9, no. 12, p. 2106, 2020.
- [26] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2020.
- [27] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proc. 57th Annu. Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 3645–3650.
- [28] P. Henderson, J. Hu, J. Romoff, E. Brunskill, D. Jurafsky, and J. Pineau, "Towards the systematic reporting of the energy and carbon footprints of machine learning," *Journal of Machine Learning Research*, vol. 21, no. 248, pp. 1–43, 2020. [Online]. Available: <https://jmlr.org/papers/v21/20-312.html>
- [29] L. A. Barroso and U. Hölzle, "The case for energy-proportional computing," *Computer*, vol. 40, no. 12, pp. 33–37, 2007.
- [30] R. G. Kula, D. M. German, A. Ouni, T. Ishio, and K. Inoue, "Do developers update their library dependencies?" *Empirical Software Engineering*, vol. 23, no. 1, pp. 384–417, 2018.
- [31] A. Zerouali, T. Mens, G. Robles, and J. M. Gonzalez-Barahona, "On the relation between outdated Docker containers, severity

- vulnerabilities, and bugs,” in *Proc. IEEE 26th Int. Conf. Software Analysis, Evolution and Reengineering (SANER)*, 2019, pp. 491–501.
- [32] R. Shu, X. Gu, and W. Enck, “A study of security vulnerabilities on Docker Hub,” in *Proc. 7th ACM Conf. Data and Application Security and Privacy (CODASPY)*, 2017, pp. 269–280.
 - [33] T. Makkouk, D. J. Kim, and T.-H. P. Chen, “An empirical study on performance bugs in deep learning frameworks,” in *Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME)*, 2022, pp. 35–46.
 - [34] M. Shahriari, R. Ramler, and L. Fischer, “How do deep-learning framework versions affect the reproducibility of neural network models?” *Machine Learning and Knowledge Extraction*, vol. 4, no. 4, pp. 888–911, 2022.
 - [35] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, “Producing wrong data without doing anything obviously wrong!” in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2009, pp. 265–276.
 - [36] A. Georges, D. Buytaert, and L. Eeckhout, “Statistically rigorous Java performance evaluation,” in *Proc. 22nd Annu. ACM SIGPLAN Conf. Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, 2007, pp. 57–76.
 - [37] T. Kalibera and R. Jones, “Rigorous benchmarking in reasonable time,” in *Proc. Int. Symp. Memory Management (ISMM)*. ACM, 2013, pp. 63–74.
 - [38] T. Hoefler and R. Belli, “Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results,” in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, 2015.
 - [39] S. H. Hurlbert, “Pseudoreplication and the design of ecological field experiments,” *Ecological Monographs*, vol. 54, no. 2, pp. 187–211, 1984.
 - [40] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.
 - [41] ONNX Runtime developers, “ONNX Runtime,” <https://onnxruntime.ai>, 2018, source code: <https://github.com/microsoft/onnxruntime>; MIT License; first released 2018-11-29.
 - [42] D. S. Lin, “Greenhouse imagery and deployment telemetry for a containerised edge vision node on a Raspberry Pi 5 (version 2.0.0),” Zenodo dataset, 2026. [Online]. Available: <https://doi.org/10.5281/zenodo.23007169>

Lin Ding Shan is an undergraduate student in data science with the Faculty of Computer Science and Data Science, UCSI University, Kuala Lumpur, Malaysia, expected to graduate in 2027. He has built and deployed offline computer-vision systems on Raspberry Pi 5 edge devices for agricultural monitoring, including a basil disease-monitoring node in a commercial greenhouse and a durian disease and pest scouting device. His research interests include edge AI deployment and the latency, energy and thermal behaviour of inference on embedded devices.